UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

Spotify

Segmentación de canciones, predicción de popularidad, clasificación de género musical y reglas de asociación

Proyecto Final

Materia: Minería de Datos | **Grupo:** 032

Profesora: Valeria López García

Integrantes del equipo

Luis Rolando Aguilar Ortiz - 2121179
Yael Abiezer García Guerra - 2050107
Perla Natalie González Treviño - 1903796
Axel Ivan Verastegui Patiño - 2091219
Jaime Cesar Zermeño Pereyra - 2121015

Monterrey, Nuevo León | 11 de Noviembre de 2026

Índice

1. Resumen **3**
2. Introducción **3**
3. Contexto **4**
3.1 ¿Qué es la minería de datos? 4
3.2 Principales técnicas 4
3.3 Ventajas de la minería de datos y utilidad para este caso 5
4. Caso de estudio **6**
4.1 Descripción del negocio 6
4.2 Antecedentes 6
4.3 Los cuatro problemas 6
5. Justificación de técnicas **7**
5.1 Agrupamiento con K-Means y reducción con PCA (problema 1) 7
5.2 Regresión con Random Forest y Gradient Boosting (problema 2) 8
5.3 Clasificación con Random Forest y SVM (problema 3) 8
5.4 Reglas de asociación con Apriori (problema 4) 8
6. Tipos de análisis **8**
7. Proceso de minería de datos **10**
7.1 Algoritmos y herramientas 10
7.2 Recolección de datos 11
7.3 Data set 11
7.4 Pasos del análisis 12
7.5 Análisis adicionales 35
8. Resultados **37**
8.1 Resultados obtenidos 37
8.2 Cómo se podrían aprovechar los resultados 38
8.3 Otras opciones y soluciones aplicables 38
9. Conclusiones **39**
9.1 Conclusión general 39
9.2 Conclusiones individuales 39
10. Referencias **41**

1. Resumen

Este proyecto aplica técnicas de minería de datos al *Spotify Tracks Dataset* (114,000 registros de canciones de 114 géneros, con 21 columnas) para resolver cuatro problemas: segmentar las canciones por su sonido, predecir su popularidad, clasificar su género musical y encontrar reglas de asociación entre las características de audio y la popularidad alta. Después de limpiar los datos (113,836 registros válidos y 89,578 canciones distintas) se utilizaron K-Means con PCA, Random Forest, Gradient Boosting, SVM y Apriori en Python.

Los resultados muestran que el audio permite formar cinco segmentos de canciones fáciles de interpretar y reconocer géneros muy distintos entre sí (57.2% de accuracy con 10 géneros), pero explica poco la popularidad (R^2 de 0.09 con audio y metadatos, y de 0.285 al agregar el género) y las reglas de asociación hacia la popularidad alta son débiles (lift máximo de 1.39). Se concluye que el audio sirve para describir y organizar un catálogo, pero que predecir el éxito de una canción requiere datos adicionales, como el artista, la promoción o el comportamiento de los usuarios.

Palabras clave: Minería de datos, Spotify, K-Means, Random Forest, SVM, Apriori, popularidad, recomendación.

2. Introducción

Spotify es una de las plataformas de música en streaming más grandes del mundo: al cierre del tercer trimestre de 2026 reportó 788 millones de usuarios activos mensuales y 306 millones de suscriptores Premium (Spotify, 2026). Cada reproducción, cada búsqueda y cada canción del catálogo genera datos, y a partir de esos datos la plataforma decide qué recomendar, cómo ordenar sus listas y qué contenido promover. Sin embargo, esa cantidad de información es imposible de revisar a mano, y por eso surge la necesidad de usar técnicas de minería de datos.

Este análisis se creó porque el catálogo de una plataforma como Spotify plantea preguntas muy concretas: ¿qué canciones se parecen entre sí?, ¿se puede saber de antemano qué tan popular será una canción?, ¿es posible reconocer el género de una canción solo por cómo suena?, ¿qué rasgos acompañan a los éxitos? Para contestarlas se utilizó el conjunto de datos público *Spotify Tracks Dataset* (Pandya, s.f.), que contiene 114,000 registros con las características de audio que la propia plataforma calcula para cada canción.

Objetivo general. Aplicar técnicas de minería de datos al conjunto de datos de canciones de Spotify para extraer conocimiento útil sobre su catálogo. De forma específica se busca: (1) segmentar las canciones en grupos de sonido similar para construir un sistema de recomendación basado en contenido; (2) predecir el puntaje de popularidad de una canción; (3) clasificar el género musical a partir de características acústicas; y (4) encontrar reglas de asociación entre las características de audio y la popularidad alta.

Por qué minería de datos. Los datos son numerosos (más de cien mil registros), tienen muchas variables que interactúan entre sí y no existe una regla sencilla que relacione el sonido de una canción con su género o su popularidad. La minería de datos permite descubrir esos patrones de manera automática, tanto sin etiquetas (agrupamiento y asociación) como con ellas (regresión y clasificación), y además medir objetivamente qué tan bien funcionan.

Beneficios y resultados esperados. Se espera obtener grupos de canciones interpretables que sirvan para recomendar música parecida, saber hasta qué punto se puede anticipar la popularidad con los datos disponibles, contar con un clasificador de género que ayude a etiquetar canciones nuevas y conocer qué combinaciones de rasgos aparecen más en las canciones populares. Para cada resultado se explica además cómo podría aplicarse en un negocio de streaming.

El documento sigue el orden del proceso de minería de datos: primero se presentan el contexto y el caso de estudio, después las técnicas elegidas y su justificación, luego el proceso paso a paso con imágenes del Jupyter Notebook y, finalmente, los resultados, las conclusiones del equipo y las referencias.

3. Contexto

3.1 ¿Qué es la minería de datos?

La minería de datos es el proceso de descubrir patrones, relaciones y conocimiento útil y no evidente dentro de grandes volúmenes de datos (Fayyad et al., 1996; Han et al., 2012). Forma parte de un proceso más amplio llamado descubrimiento de conocimiento en bases de datos (KDD), que incluye la selección, limpieza y transformación de los datos antes de aplicar los algoritmos, y la interpretación de los resultados después. La metodología CRISP-DM (Chapman et al., 2000) organiza este trabajo en seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Este proyecto sigue esa misma secuencia.

3.2 Principales Técnicas

Las técnicas de minería de datos se dividen en **supervisadas** (aprenden a partir de ejemplos con una respuesta conocida) y **no supervisadas** (buscan estructura en datos sin respuesta). La Tabla 1 resume las que se usan en este proyecto.

Tabla 1. *Principales técnicas de minería de datos y su uso en el proyecto*

TÉCNICA	TIPO	QUÉ HACE	USO EN EL PROYECTO
Agrupamiento (clustering)	No supervisada	Forma grupos de registros parecidos entre sí	Segmentar canciones por su sonido (K-Means)
Reducción de dimensionalidad	No supervisada	Resume muchas variables en pocas, conservando la información	Visualizar los grupos en 2 dimensiones (PCA)
Reglas de asociación	No supervisada	Encuentra elementos que aparecen juntos con frecuencia	Rasgos de audio asociados a la popularidad alta (Apriori)
Regresión	Supervisada	Predice un valor numérico	Predecir el puntaje de popularidad
Clasificación	Supervisada	Predice una categoría	Predecir el género musical

3.3 Ventajas de la minería de datos y utilidad para este caso

La minería de datos se usa en muchas áreas porque permite tomar decisiones con evidencia en lugar de intuición. En el **sector financiero** ayuda a detectar transacciones fraudulentas y a estimar el riesgo de crédito; en **salud**, a identificar patrones asociados a enfermedades y a priorizar pacientes; en el **comercio**, a descubrir qué productos se compran juntos y a segmentar clientes; y en el **entretenimiento**, a recomendar contenido personalizado, que es justamente lo que hacen plataformas como Spotify.

Para el caso de estudio resulta especialmente útil por tres razones: (1) *el catálogo es demasiado grande para analizarlo manualmente*; (2) *cada canción ya viene descrita por variables numéricas objetivas (bailabilidad, energía, tempo, etc.), lo que facilita aplicar los algoritmos;* y (3) *los resultados tienen aplicación directa en la recomendación de música, la organización de listas de reproducción y la planeación de lanzamientos*.

4. Caso de estudio

4.1 Descripción del negocio

Spotify es un servicio de streaming de música y podcasts con un modelo *freemium*: los usuarios pueden escuchar gratis con anuncios o pagar una suscripción Premium. Sus ingresos provienen de ambas fuentes, por lo que su negocio depende de dos cosas: que los usuarios encuentren música que les guste (para que sigan usando la plataforma y se conviertan en suscriptores) y que el catálogo se organice y se promueva de forma inteligente. Con 788 millones de usuarios activos y 306 millones de suscriptores Premium (Spotify, 2026), una mejora pequeña en la recomendación impacta a millones de personas.

4.2 Antecedentes

Para cada canción, la plataforma calcula a través de su API Web un conjunto de **características de audio** (por ejemplo, qué tan bailable, enérgica o acústica es) y un puntaje de **popularidad** que se basa principalmente en el número total de reproducciones y en qué tan recientes son (Pandya, s.f.). Un usuario independiente reunió estos datos de 114,000 canciones de 114 géneros en el conjunto *Spotify Tracks Dataset*, con exactamente 1,000 canciones por género. Ese conjunto es el que se analiza aquí. Dado que contiene datos de canciones y no de usuarios, el análisis se enfoca en el **contenido** de las canciones y no en el comportamiento de los oyentes.

4.3 Los cuatro problemas

A partir del conjunto de datos se plantearon cuatro problemas. En la Tabla 2 se resumen las variables que interviene en cada uno y a continuación se describe cada problema con detalle.

Tabla 2. *Variables y tipo de análisis de cada problema*

PROBLEMA	VARIABLES DE ENTRADA	VARIABLE DE SALIDA	TIPO DE ANÁLISIS
1. Segmentación	9 características de audio continuas	Grupo (cluster) de cada canción	Descriptivo, no supervisado
2. Popularidad	14 características de audio y metadatos (y el género en una variante)	popularity (0 a 100)	Predictivo, supervisado (regresión)
3. Género musical	12 características acústicas	track_genre	Predictivo, supervisado (clasificación)
4. Asociación	9 características de audio discretizadas y explicit	Popularidad alta o baja	Descriptivo, no supervisado

Problema 1: Segmentación De Canciones Por Características De Audio. Un catálogo con decenas de miles de canciones necesita agruparse de manera que canciones que suenan parecido queden juntas, sin importar su género o artista. Con 9 variables continuas (danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence y tempo) se busca formar grupos, describirlos y usarlos para recomendar canciones similares a una canción dada.

Problema 2: Predicción De La Popularidad. Para una disquera o para la plataforma sería muy valioso anticipar qué tan popular será una canción. Aquí la variable a predecir es `popularity`, un número de 0 a 100, a partir de las características de audio, la duración, si es explícita o no y, en una segunda variante, el género. Se quiere saber cuánto de la popularidad se explica por el sonido y cuánto queda sin explicar.

Problema 3: Clasificación Del Género Musical. Etiquetar manualmente el género de cada canción nueva es lento y subjetivo. Se busca predecir `track_genre` usando únicamente características acústicas (sin popularidad, duración, artista ni nombre), y evaluar en qué géneros el sonido es suficiente para distinguirlos y en cuáles se confunden.

Problema 4: Asociación Entre Audio Y Popularidad Alta. En lugar de predecir un número, se buscan combinaciones de rasgos (por ejemplo, "bailabilidad alta y energía alta") que aparezcan con frecuencia en las canciones más populares y, en contraste, en las menos populares. Los resultados se expresan como reglas fáciles de interpretar.

Identificación de variables. Todas las variables numéricas del conjunto de datos están descritas en la sección 7.3. Para los problemas 1, 3 y 4 la salida se obtiene sin una respuesta previa (1 y 4) o es el género (3); en el problema 2 la variable objetivo es la popularidad. En el problema 4 la popularidad se transforma en una categoría (alta, media, baja) y las variables continuas se dividen en niveles (baja, media, alta) porque el algoritmo Apriori requiere variables categóricas.

5. Justificación de técnicas

Para cada problema se eligió la técnica que mejor corresponde a su pregunta de negocio. A continuación se define cada una y se explica por qué se decidió utilizarla.

5.1 Agrupamiento con K-Means y reducción con PCA (problema 1)

Definición. El agrupamiento es una técnica no supervisada que divide los datos en grupos de manera que los elementos de un mismo grupo se parezcan más entre sí que a los de otros grupos. K-Means (MacQueen, 1967) coloca *k* centroides y asigna cada registro al más cercano, repitiendo el proceso hasta que los grupos se estabilizan. El número de grupos se decide con el método del codo (WCSS) y con el coeficiente de silueta (Rousseeuw, 1987). El análisis de componentes principales (PCA) transforma las variables originales en nuevos ejes no correlacionados que conservan la mayor varianza posible (Jolliffe & Cadima, 2016).

Por qué. No existe una etiqueta que indique a qué grupo pertenece cada canción, así que se necesita una técnica no supervisada. K-Means es sencillo, rápido con casi 90,000 canciones y sus grupos son fáciles de describir con promedios. PCA se usa solo para poder dibujar los grupos en dos dimensiones. Los grupos sirven como base de un recomendador basado en contenido.

5.2 Regresión con Random Forest y Gradient Boosting (problema 2)

Definición. La regresión es una técnica supervisada que predice un valor numérico. Un Random Forest (Breiman, 2001) combina muchos árboles de decisión entrenados con muestras distintas y promedia sus predicciones, lo que reduce el sobreajuste. El Gradient Boosting (Friedman, 2001) construye los árboles uno tras otro, de modo que cada uno corrige los errores del anterior; XGBoost (Chen & Guestrin, 2016) es una implementación muy popular de esta idea.

Por qué. La popularidad es un número, por lo que corresponde una regresión. Se eligieron modelos de árboles porque capturan relaciones no lineales e interacciones entre variables sin necesidad de especificarlas, y se compararon contra una regresión lineal y contra un modelo base que siempre predice el promedio, para saber si realmente aportan algo.

5.3 Clasificación con Random Forest y SVM (problema 3)

Definición. La clasificación es una técnica supervisada que predice una categoría. Además del Random Forest (ahora como clasificador, donde los árboles votan), se usó una máquina de vectores de soporte (SVM, Cortes & Vapnik, 1995), que busca la frontera que mejor separa las clases y, con un kernel RBF, puede trazar fronteras no lineales.

Por qué. El género es una categoría y se cuenta con una etiqueta por canción, por lo que se trata de aprendizaje supervisado. Random Forest funciona bien con variables de escalas distintas y da la importancia de cada característica; SVM suele ser muy competitivo cuando hay pocas variables numéricas, por lo que es una buena comparación.

5.4 Reglas de asociación con Apriori (problema 4)

Definición. Las reglas de asociación descubren elementos que aparecen juntos con frecuencia. El algoritmo Apriori (Agrawal & Srikant, 1994) encuentra primero los conjuntos de elementos frecuentes (con un *support* mínimo) y de ellos genera reglas del tipo "si A entonces B", que se miden con *support*, *confidence* y *lift*.

Por qué. Los modelos de regresión dan un número, pero no dicen qué combinaciones de rasgos se relacionan con el éxito. Las reglas son fáciles de interpretar y comunicar, y permiten ver también qué combinaciones acompañan a la popularidad baja. Es el mismo enfoque que se usa en el análisis de canasta de mercado, aplicado aquí a rasgos musicales.

6. Tipos de análisis

En el proyecto se realizan dos tipos de análisis diferentes y complementarios: uno **descriptivo**, que resume y explica cómo es el catálogo, y uno **predictivo**, que construye modelos para anticipar valores o categorías de canciones nuevas. Además, el tema de la popularidad se estudia desde ambos enfoques: se predice con regresión (análisis predictivo) y se describe con reglas de asociación (análisis descriptivo).

Tabla 3. *Tipos de Análisis Realizados*

ANÁLISIS	TIPO	APRENDIZAJE	MODELOS	CÓMO SE EVALÚA
1. Segmentación	Descriptivo	No supervisado	K-Means + PCA	Codo (WCSS), silueta y perfil de cada grupo
2. Popularidad	Predictivo	Supervisado (regresión)	Regresión lineal, Random Forest, Gradient Boosting	R^2, MAE y RMSE en datos de prueba
3. Género	Predictivo	Supervisado (clasificación)	Random Forest, SVM	Accuracy, F1 y matriz de confusión
4. Asociación	Descriptivo	No supervisado	Apriori	Support, confidence y lift

7. Proceso de minería de datos

Esta sección documenta cómo se llevó a cabo el análisis: las herramientas y algoritmos, el origen de los datos, el conjunto de datos y cada paso del Jupyter Notebook, explicado con imágenes.

7.1 Algoritmos y herramientas

Todo el trabajo se realizó en **Python** dentro de **Jupyter Notebook**, con las librerías que se listan en la Tabla 4. El notebook completo (`proyecto_final_spotify.ipynb`) se entrega junto con este documento y contiene el código y los resultados de cada paso.

Tabla 4. *Herramientas Utilizadas*

HERRAMIENTA	PARA QUÉ SE USÓ
Python y Jupyter Notebook	Entorno de programación y de documentación de los pasos del análisis
pandas (McKinney, 2010)	Carga, limpieza, agrupación y resumen de los datos
scikit-learn (Pedregosa et al., 2011)	Estandarización, K-Means, PCA, regresión, clasificación, validación y métricas
Matplotlib (Hunter, 2007) y seaborn (Waskom, 2021)	Gráficas e imágenes de resultados
Código propio de Apriori	Algoritmo Apriori programado en el notebook (la librería mlxtend no estaba disponible)

En la Tabla 5 se muestran los algoritmos y sus parámetros principales. Cabe aclarar que en la actividad 3.1 se propuso XGBoost como alternativa para predecir la popularidad; como esa librería no estaba instalada en el entorno de trabajo, se utilizó `HistGradientBoostingRegressor` de scikit-learn, que implementa la misma técnica de *gradient boosting* con árboles. De igual forma, Apriori se programó directamente en el notebook.

Tabla 5. *Algoritmos y Parámetros Principales*

ANÁLISIS	ALGORITMO (CLASE O FUNCIÓN)	PARÁMETROS PRINCIPALES
1	KMeans, PCA, StandardScaler	k elegido con codo y silueta; n_init = 10; 2 componentes; random_state = 0
2	RandomForestRegressor, HistGradientBoostingRegressor, LinearRegression	100 árboles, min_samples_leaf = 20; separación 80/20 por grupos (GroupShuffleSplit)
3	RandomForestClassifier, SVC (kernel RBF)	RF: 100 y 300 árboles; SVM: C y gamma con GridSearchCV de 3 pliegues
4	Apriori propio	min_support = 0.02; itemsets de hasta 4 elementos; consecuente = popularidad

Bases de datos de consulta. Se consultó el conjunto *Spotify Tracks Dataset* (Pandya, s.f.) y su documentación, donde se describe cada variable. No se usó ninguna base de datos relacional; el análisis parte de un archivo CSV.

7.2 Recolección de datos

Los datos fueron recolectados por el autor del conjunto a través de la API Web de Spotify, que devuelve para cada canción sus metadatos y sus características de audio (Pandya, s.f.). El equipo trabajó con el archivo `dataset.csv` (aproximadamente 20 MB) del conjunto *Spotify_Kaggle* indicado en la actividad 3.1, que tiene las mismas columnas y estructura que el conjunto documentado por Pandya (s.f.) en Hugging Face. El archivo no se modificó: toda la limpieza se hizo desde el notebook, de manera reproducible. Es importante señalar que la documentación del conjunto menciona 125 géneros, pero el archivo que se analizó contiene 114.

7.3 Data set

El conjunto de datos tiene **114,000 registros y 21 columnas**. Cada registro es una canción dentro de un género. En las Figuras 1 y 2 se muestran las primeras filas del archivo, separadas en datos descriptivos y características de audio para que se alcancen a leer.

Figura 1. *Primeras filas del data set: identificación y metadatos*

	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	track_genre
0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	acoustic
1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	acoustic
2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	acoustic
3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Origina…	Can't Help Falling In Love	71	201933	False	acoustic
4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	acoustic

Figura 2. *Primeras filas del data set: características de audio*

	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	time_signature
0	0.676	0.4610	1	-6.746	0	0.1430	0.0322	0.000001	0.3580	0.715	87.917	4
1	0.420	0.1660	1	-17.235	1	0.0763	0.9240	0.000006	0.1010	0.267	77.489	4
2	0.438	0.3590	0	-9.734	1	0.0557	0.2100	0.000000	0.1170	0.120	76.332	4
3	0.266	0.0596	0	-18.515	1	0.0363	0.9050	0.000071	0.1320	0.143	181.740	3
4	0.618	0.4430	2	-9.681	1	0.0526	0.4690	0.000000	0.0829	0.167	119.949	4

La Tabla 6 describe todos los atributos (columnas). En la última columna se indica en qué análisis se usa cada uno (1 = segmentación, 2 = popularidad, 3 = género, 4 = asociación).

Tabla 6. *Atributos del data set*

COLUMNA	TIPO	DESCRIPCIÓN	USO
Unnamed: 0	Entero	Índice del registro (se elimina)	-
track_id	Texto	Identificador de la canción en Spotify	Duplicados
artists	Texto	Artista(s) que interpretan la canción	Recomendación
album_name	Texto	Álbum en el que aparece	Informativa
track_name	Texto	Nombre de la canción	Recomendación
popularity	Entero (0 a 100)	Popularidad según reproducciones totales y recientes	Objetivo (2); nivel (4)
duration_ms	Entero	Duración en milisegundos	2
explicit	Booleano	Si la letra es explícita	2, 4
danceability	Real (0 a 1)	Qué tan adecuada es para bailar	1, 2, 3, 4
energy	Real (0 a 1)	Intensidad y actividad percibidas	1, 2, 3, 4
key	Entero (0 a 11)	Tono de la canción (0 = Do, 1 = Do#, etc.)	2, 3
loudness	Real (dB)	Volumen promedio de la pista	1, 2, 3, 4
mode	Entero (0 o 1)	Modo: mayor (1) o menor (0)	2, 3
speechiness	Real (0 a 1)	Presencia de palabras habladas	1, 2, 3, 4
acousticness	Real (0 a 1)	Confianza de que la pista es acústica	1, 2, 3, 4

COLUMNA	TIPO	DESCRIPCIÓN	USO
instrumentalness	Real (0 a 1)	Probabilidad de que no tenga voz	1, 2, 3, 4
liveness	Real (0 a 1)	Probabilidad de que sea una grabación en vivo	1, 2, 3, 4
valence	Real (0 a 1)	Positividad musical (alegre o triste)	1, 2, 3, 4
tempo	Real (BPM)	Velocidad estimada en pulsos por minuto	1, 2, 3, 4
time_signature	Entero	Compás estimado (pulsos por compás)	2, 3
track_genre	Texto (114 géneros)	Género de la canción	Etiqueta (3); variante (2)

7.4 Pasos del análisis

A continuación se muestran los pasos del Jupyter Notebook mediante imágenes del código y de sus resultados, junto con la explicación de cada uno. Las gráficas son las que genera el propio notebook.

7.4.1 Carga y calidad de los datos

Paso 1. Importar librerías y cargar los datos. Se importan pandas, NumPy, Matplotlib y seaborn y se lee el archivo `dataset.csv` en un DataFrame. La salida de `shape` confirma que se cargaron 114,000 filas y 21 columnas.

Figura 3. *Importación de librerías y carga del data set*

```
In [1]:
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')
plt.rcParams.update({'font.size': 11, 'axes.titlesize': 12, 'figure.dpi':
100})

In [2]:
dataset = pd.read_csv('dataset.csv')

In [4]:
dataset.shape

Out[4]:
(114000, 21)
```

Paso 2. Revisar el tipo de dato, los nulos y los valores únicos. Solo `artists`, `album_name` y `track_name` tienen un valor nulo (la misma fila). Hay 114 géneros y 89,741 canciones distintas (`track_id`), por lo que hay canciones que aparecen en más de un género.

Figura 4. *Calidad de los datos: tipos, nulos y valores únicos*

```
In [5]:
dataset = dataset.drop(columns='Unnamed: 0')
calidad = pd.DataFrame({'tipo': dataset.dtypes.astype(str),
'nulos': dataset.isna().sum(), 'valores_unicos':
dataset.nunique()})
calidad
```

Out[5]:

	tipo	nulos	valores_unicos
track_id	str	0	89741
artists	str	1	31437
album_name	str	1	46589
track_name	str	1	73608
popularity	int64	0	101
duration_ms	int64	0	50697
explicit	bool	0	2
danceability	float64	0	1174
energy	float64	0	2083
key	int64	0	12
loudness	float64	0	19480
mode	int64	0	2
speechiness	float64	0	1489
acousticness	float64	0	5061
instrumentalness	float64	0	5346
liveness	float64	0	1722
valence	float64	0	1790
tempo	float64	0	45653
time_signature	int64	0	5
track_genre	str	0	114

Paso 3. Buscar duplicados y mediciones inválidas. Se detectan 24,259 `track_id` repetidos, 157 registros con tempo igual a 0, 163 con compás igual a 0 y uno con duración 0 (mediciones que la API no pudo calcular). Cada género tiene exactamente 1,000 registros y 16,020 canciones tienen popularidad 0.

Figura 5. *Revisión de duplicados y valores inválidos*

```
In [7]:
print('Filas duplicadas exactas:', dataset.duplicated().sum())
print('track_id repetidos:', dataset['track_id'].duplicated().sum())
print('Canciones distintas (track_id):', dataset['track_id'].nunique())
print('Generos distintos:', dataset['track_genre'].nunique())
print('Canciones por genero (min, max):',
dataset['track_genre'].value_counts().min(),
dataset['track_genre'].value_counts().max())
print('Tempo = 0:', (dataset['tempo'] == 0).sum(), '| time_signature =
0:', (dataset['time_signature'] == 0).sum(), '| duracion = 0:',
(dataset['duration_ms'] == 0).sum())
print('Popularidad = 0:', (dataset['popularity'] == 0).sum())

Filas duplicadas exactas: 450
track_id repetidos: 24259
Canciones distintas (track_id): 89741
Generos distintos: 114
Canciones por genero (min, max): 1000 1000
Tempo = 0: 157 | time_signature = 0: 163 | duracion = 0: 1
Popularidad = 0: 16020
```

7.4.2 Limpieza y preparación

Paso 4. Limpiar y crear las dos versiones de los datos. Se elimina la fila con nulos y las mediciones inválidas, `explicit` se convierte a 0 y 1 y se crea una llave `cancion` (artista y nombre) para identificar canciones repetidas. Quedan **113,836 registros** (canción-género) y **89,578 canciones distintas**. La versión `dataset` se usa en los análisis supervisados y `canciones` en la segmentación y en las reglas de asociación.

Figura 6. *Limpieza y preparación de los datos*

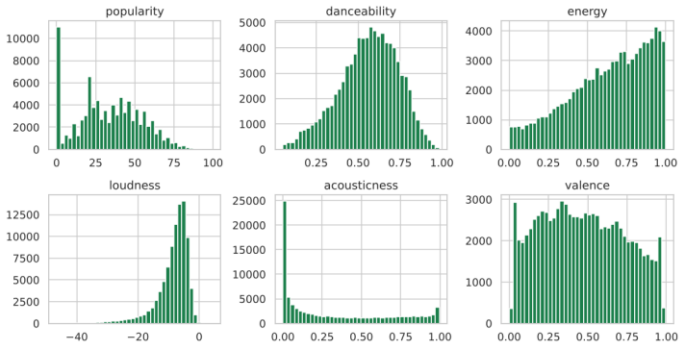
```
In [8]: dataset = dataset.dropna()
dataset = dataset[(dataset['tempo'] > 0) & (dataset['time_signature'] >
0) & (dataset['duration_ms'] > 0)].reset_index(drop=True)
dataset['explicit'] = dataset['explicit'].astype(int)
dataset['cancion'] = dataset['artists'].str.lower() + ' | ' +
dataset['track_name'].str.lower()
canciones = dataset.drop_duplicates('track_id').reset_index(drop=True)
print('Registros (cancion-genero):', len(dataset))
print('Canciones distintas:', len(canciones))

Registros (cancion-genero): 113836
Canciones distintas: 89578
```

7.4.3 Análisis exploratorio (EDA)

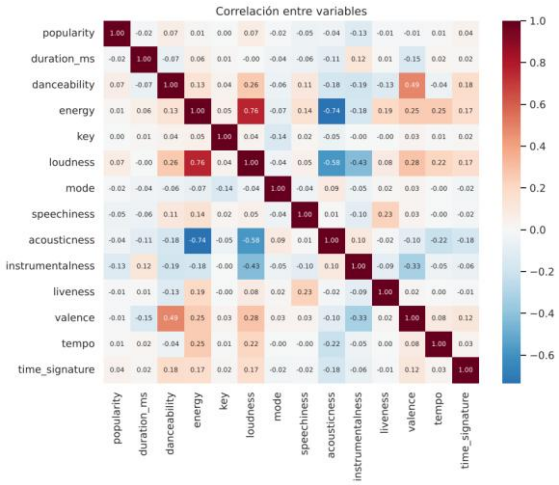
Paso 5. Distribución de las variables. La popularidad se concentra en valores bajos (hay un pico en 0), `danceability` se parece a una campana, `energy` y `loudness` se cargan hacia valores altos y `acousticness` hacia valores bajos.

Figura 7. *Histogramas de popularidad y de cinco características de audio*



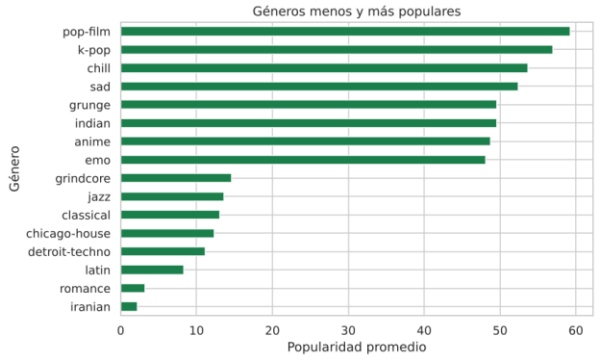
Paso 6. Correlación entre variables. La popularidad tiene correlaciones muy débiles con todas las variables (la mayor es -0.13 con `instrumentalness`). En cambio, `energy` está muy relacionada con `loudness` (0.76) y con `acousticness` (-0.74).

Figura 8. *Matriz de correlación*



Paso 7. Popularidad por género. Los géneros menos populares son *iranian* (2.2), *romance* (3.2) y *latin* (8.3) y los más populares son *pop-film* (59.3), *k-pop* (57.0) y *chill* (53.7). El género es, por tanto, muy importante para la popularidad.

Figura 9. *Géneros con menor y mayor popularidad promedio*



7.4.4 Análisis 1: segmentación con K-Means y PCA

Paso 8. Estandarizar las 9 características de audio. Se usan `danceability`, `energy`, `loudness`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence` y `tempo`. Con `StandardScaler` cada variable queda con media 0 y desviación 1, porque K-Means usa distancias y `tempo` y `loudness` tienen escalas muy distintas.

Figura 10. *Estandarización de las características*

```
In [12]: from sklearn.preprocessing import StandardScaler
caracteristicas = ['danceability', 'energy', 'loudness', 'speechiness',
'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo']
escalador = StandardScaler()
X_seg = escalador.fit_transform(canciones[caracteristicas])
X_seg.shape
Out[12]: (89578, 9)
```

Paso 9. Elegir el número de clusters. Se prueba K-Means con k de 2 a 10 y se calculan la WCSS (método del codo) y el coeficiente de silueta con una muestra de 10,000 canciones.

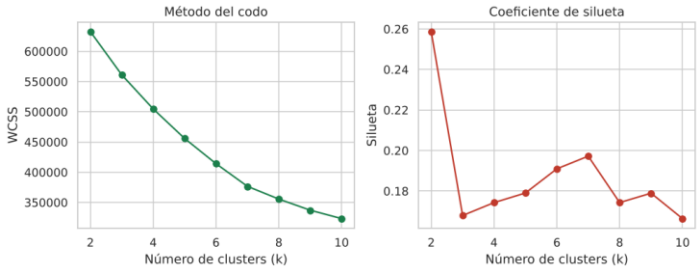
Figura 11. *Código y resultados del método del codo y la silueta*

```
In [13]: from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
muestra = np.random.RandomState(0).choice(len(X_seg), 10000,
replace=False)
wcss, silueta = [], []
for k in range(2, 11):
    modelo_k = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X_seg)
    wcss.append(modelo_k.inertia_)
    silueta.append(silhouette_score(X_seg[muestra],
modelo_k.labels_[muestra]))
pd.DataFrame({'k': range(2, 11), 'WCSS': np.round(wcss, 0), 'Silueta':
np.round(silueta, 3)})
```

Out[13]:

	k	WCSS	Silueta
0	2	632568.0	0.259
1	3	561036.0	0.168
2	4	504696.0	0.174
3	5	455847.0	0.179
4	6	414121.0	0.191
5	7	376636.0	0.197
6	8	355561.0	0.174
7	9	337114.0	0.179
8	10	323249.0	0.166

Figura 12. *Método del codo y coeficiente de silueta*



La WCSS baja de manera continua y no hay un codo claro. La silueta es mayor con k = 2 (0.259), pero con solo dos grupos la separación es demasiado general. Se eligió **k = 5** (silueta de 0.179) porque produce segmentos con una interpretación musical clara. Con k = 6 aparece un grupo de grabaciones en vivo y con k = 7 uno de música instrumental intensa.

Paso 10. Entrenar K-Means y reducir con PCA. Con k = 5 los grupos quedan de 1,098, 33,735, 20,655, 26,824 y 7,266 canciones. Los dos primeros componentes de PCA explican el 47.9% de la varianza (PC1 = 31.9%, PC2 = 16.0%).

Figura 13. *Entrenamiento de K-Means y PCA*

```
In [15]:
kmeans = KMeans(n_clusters=5, n_init=10, random_state=0)
canciones['cluster'] = kmeans.fit_predict(X_seg)
canciones['cluster'].value_counts().sort_index()
```

```
Out[15]:
cluster
0     1098
1    33735
2    20655
3    26824
4     7266
Name: count, dtype: int64
```

```
In [19]:
from sklearn.decomposition import PCA
pca = PCA(n_components=2, random_state=0)
componentes = pca.fit_transform(X_seg)
print('Varianza explicada por PC1 y PC2:',
pca.explained_variance_ratio_.round(3), '| total:',
pca.explained_variance_ratio_.sum().round(3))
pd.DataFrame(pca.components_.T, index=caracteristicas, columns=['PC1',
'PC2']).round(2)
```

```
Varianza explicada por PC1 y PC2: [0.319 0.16 ] | total: 0.479
```

Out[19]:

	PC1	PC2
danceability	0.25	0.57
energy	0.51	-0.28
loudness	0.52	-0.08
speechiness	0.10	0.04
acousticness	-0.44	0.31
instrumentalness	-0.28	-0.32
liveness	0.09	-0.21
valence	0.30	0.53
tempo	0.19	-0.28

Paso 11. Perfil de los segmentos. Se calcula el promedio de cada característica por grupo. Con base en el perfil se les dio nombre: **Hablado** (`speechiness` 0.83), **Bailable y alegre** (`danceability` y `valence` de 0.69), **Suave y acústico** (`energy` 0.39, `acousticness` 0.67), **Intenso y oscuro** (`energy` 0.82, `valence` 0.32, tempo 136) e **Instrumental y ambiental** (`instrumentalness` 0.79, volumen muy bajo). Los géneros más frecuentes de cada grupo (comedy, salsa y dancehall, tango, grindcore y black-metal, new-age y sleep) confirman la interpretación, aunque el género no se usó para agrupar.

Figura 14. *Perfil promedio de cada cluster*

```
In [16]:
perfil = canciones.groupby('cluster')[caracteristicas].mean().round(2)
perfil['canciones'] = canciones['cluster'].value_counts().sort_index()
perfil
```

Out[16]:

cluster	danceability	energy	loudness	speechiness	acousticness	instrumentalness	liveness	valence	tempo	canciones
0	0.57	0.68	-11.08	0.83	0.73	0.01	0.67	0.45	101.76	1098
1	0.69	0.73	-6.55	0.09	0.21	0.07	0.19	0.69	120.53	33735
2	0.53	0.39	-10.48	0.06	0.67	0.06	0.19	0.40	113.59	20655
3	0.48	0.82	-5.97	0.08	0.07	0.24	0.26	0.32	136.44	26824
4	0.37	0.20	-20.54	0.05	0.82	0.79	0.16	0.19	105.87	7266

```
In [18]:
nombres = {0: 'Hablado', 1: 'Bailable y alegre', 2: 'Suave y acústico', 3: 'Intenso y
oscuro', 4: 'Instrumental y ambiental'}
canciones['segmento'] = canciones['cluster'].map(nombres)
resumen_seg = canciones['segmento'].value_counts().to_frame('canciones')
resumen_seg['porcentaje'] = (resumen_seg['canciones'] / len(canciones) * 100).round(1)
resumen_seg
```

Out[18]:

segmento	canciones	porcentaje
Bailable y alegre	33735	37.7
Intenso y oscuro	26824	29.9
Suave y acústico	20655	23.1
Instrumental y ambiental	7266	8.1
Hablado	1098	1.2

Figura 15. *Perfil estandarizado de cada cluster*

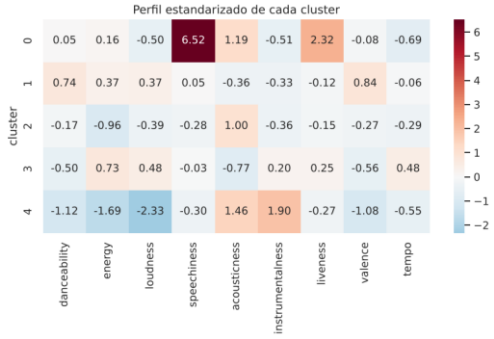
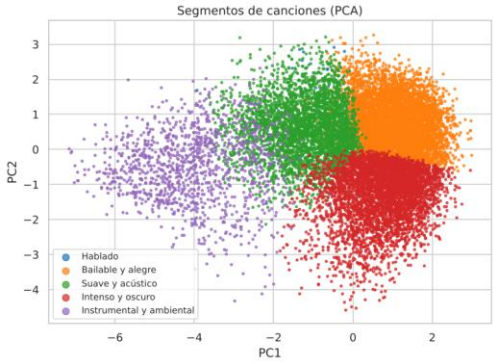


Figura 16. *Segmentos de canciones proyectados con PCA*



En la proyección, el segmento *Instrumental y ambiental* queda separado a la izquierda y los segmentos *Bailable y alegre* e *Intenso y oscuro* se tocan en la derecha; el traslape coincide con la silueta baja.

Paso 12. Recomendador basado en contenido. La función `recomendar` busca las canciones del mismo segmento y las ordena por similitud coseno. Para *Despacito*, las tres primeras recomendaciones son otras versiones de la misma canción (comprobación de que funciona) y las siguientes son canciones de otros géneros con sonido parecido.

Figura 17. *Función de recomendación y ejemplo con la canción Despacito*

In [21]:
```
from sklearn.metrics.pairwise import cosine_similarity
def recomendar(nombre, artista=None, n=5):
    candidatos = canciones[canciones['track_name'].str.lower() == nombre.lower()]
    if artista:
        candidatos =
candidatos[candidatos['artists'].str.lower().str.contains(artista.lower())]
    posicion = candidatos.index[0]
    mismo_cluster = canciones.index[canciones['cluster'] ==
canciones.loc[posicion, 'cluster']]
    similitud = cosine_similarity(X_seg[[posicion]], X_seg[mismo_cluster])[0]
    orden = mismo_cluster[np.argsort(-similitud)]
    orden = [i for i in orden if canciones.loc[i, 'cancion'] !=
canciones.loc[posicion, 'cancion']][:n]
    return canciones.loc[orden, ['track_name', 'artists', 'track_genre',
'segmento', 'popularity']]
```

In [22]:
```
recomendar('Despacito', 'fonsi')
```

Out[22]:

	track_name	artists	track_genre	segmento	popularity
57138	Despacito - Remix	Luis Fonsi;Daddy Yankee;Justin Bieber	latin	Bailable y alegre	70
57082	Despacito - Remix	Luis Fonsi;Daddy Yankee;Justin Bieber	latin	Bailable y alegre	72
57389	Despacito (Featuring Daddy Yankee)	Luis Fonsi;Daddy Yankee	latin	Bailable y alegre	61
71806	Bang Bang	K'NAAN;Adam Levine	reggae	Bailable y alegre	69
76247	No lo Esperaba	Ezio Oliva;ADSO	salsa	Bailable y alegre	28

7.4.5 Análisis 2: predicción de la popularidad

Paso 13. Separar entrenamiento y prueba por grupos. La separación es 80/20 con `GroupShuffleSplit`, usando la llave `cancion` como grupo para que una misma canción no quede en entrenamiento y en prueba a la vez (si no, el resultado se inflaría). Las variables de entrada son 14: las 12 características de audio, `duration_ms` y `explicit`.

Figura 18. *Variables y separación en entrenamiento y prueba*

```
In [23]:
from sklearn.model_selection import GroupShuffleSplit
variables = ['danceability', 'energy', 'key', 'loudness', 'mode',
'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence',
'tempo', 'time_signature', 'duration_ms', 'explicit']
X = dataset[variables]
y = dataset['popularity']
entrenamiento, prueba = next(GroupShuffleSplit(n_splits=1, test_size=0.2,
random_state=42).split(X, groups=dataset['cancion']))
X_train, X_test = X.iloc[entrenamiento], X.iloc[prueba]
y_train, y_test = y.iloc[entrenamiento], y.iloc[prueba]
print('Entrenamiento:', X_train.shape, '| Prueba:', X_test.shape)

Entrenamiento: (91265, 14) | Prueba: (22571, 14)
```

Paso 14. Modelos con audio y metadatos. Se comparan un modelo base (promedio), regresión lineal, Random Forest y Gradient Boosting. Ninguno explica la popularidad: el mejor R^2 es 0.091 y el MAE baja solo de 18.8 a 17.1 puntos.

Figura 19. *Modelos de popularidad con características de audio y metadatos*

```
In [24]:
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
def evaluar(nombre, modelo, Xe, Xp):
    inicio = time.time()
    modelo.fit(Xe, y_train)
    pred = modelo.predict(Xp)
    fila = {'Modelo': nombre, 'R2': r2_score(y_test, pred), 'MAE':
mean_absolute_error(y_test, pred), 'RMSE': mean_squared_error(y_test, pred) **
0.5, 'Segundos': time.time() - inicio}
    return fila, modelo, pred
filas, modelos = [], {}
for nombre, modelo in [('Base (promedio)', DummyRegressor()), ('Regresión
lineal', LinearRegression()),
                       ('Random Forest', RandomForestRegressor(n_estimators=100,
min_samples_leaf=20, max_features=0.5, n_jobs=2, random_state=42)),
                       ('Gradient Boosting',
HistGradientBoostingRegressor(random_state=42))]:
    fila, modelos[nombre], _ = evaluar(nombre, modelo, X_train, X_test)
    fila['Variables'] = 'Audio + metadatos'
    filas.append(fila)
pd.DataFrame(filas).round(3)
```

Out[24]:

	Modelo	R2	MAE	RMSE	Segundos	Variables
0	Base (promedio)	-0.000	18.778	22.229	0.004	Audio + metadatos
1	Regresión lineal	0.018	18.401	22.023	0.042	Audio + metadatos
2	Random Forest	0.085	17.112	21.259	16.328	Audio + metadatos
3	Gradient Boosting	0.091	17.201	21.187	0.573	Audio + metadatos

Paso 15. Modelos agregando el género. Como análisis adicional se repiten los modelos con el género convertido a variables ficticias. El Gradient Boosting sube a $R^2 = 0.285$ (MAE 13.95) y el Random Forest a 0.270, mientras que la regresión lineal casi no cambia (0.038).

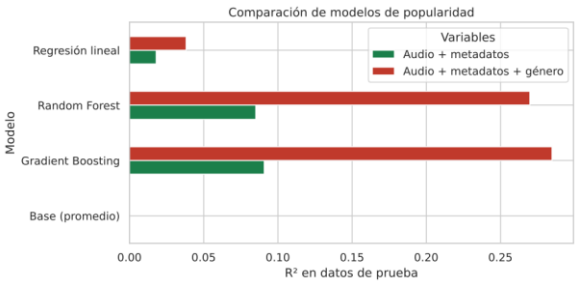
Figura 20. *Modelos de popularidad agregando el género*

```
In [25]:
generos = pd.get_dummies(dataset['track_genre']).astype('uint8')
X_gen = pd.concat([X, generos], axis=1)
Xg_train, Xg_test = X_gen.iloc[entrenamiento], X_gen.iloc[prueba]
predicciones = {}
for nombre, modelo in [('Regresión lineal', LinearRegression()),
                       ('Random Forest', RandomForestRegressor(n_estimators=100,
min_samples_leaf=20, max_features=0.33, n_jobs=2, random_state=42)),
                       ('Gradient Boosting',
HistGradientBoostingRegressor(random_state=42))]:
    fila, modelos[nombre + ' + género'], predicciones[nombre] = evaluar(nombre,
modelo, Xg_train, Xg_test)
    fila['Variables'] = 'Audio + metadatos + género'
    filas.append(fila)
resultados_pop = pd.DataFrame(filas)[['Variables', 'Modelo', 'R2', 'MAE', 'RMSE',
'Segundos']].round(3)
resultados_pop
```

Out[25]:

	Variables	Modelo	R2	MAE	RMSE	Segundos
0	Audio + metadatos	Base (promedio)	-0.000	18.778	22.229	0.004
1	Audio + metadatos	Regresión lineal	0.018	18.401	22.023	0.042
2	Audio + metadatos	Random Forest	0.085	17.112	21.259	16.328
3	Audio + metadatos	Gradient Boosting	0.091	17.201	21.187	0.573
4	Audio + metadatos + género	Regresión lineal	0.038	18.020	21.798	1.001
5	Audio + metadatos + género	Random Forest	0.270	14.041	18.988	41.514
6	Audio + metadatos + género	Gradient Boosting	0.285	13.953	18.801	3.561

Figura 21. *Comparación del R^2 de cada modelo con y sin género*



Paso 16. Importancia de variables y calidad de la predicción. El género concentra casi la mitad de la importancia (≈ 0.49) y las variables de audio tienen importancias parecidas entre sí. En la gráfica de real contra predicha se ve que el modelo se acerca al promedio: casi siempre predice entre 20 y 60 y no identifica ni las canciones con popularidad 0 ni las muy populares.

Figura 22. *Importancia de variables en el Random Forest*

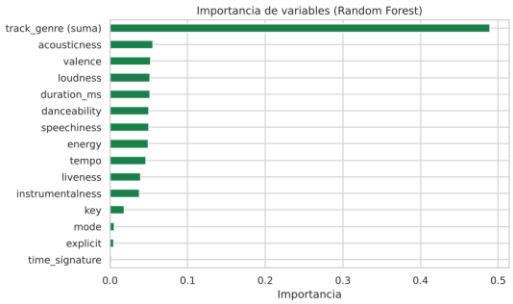
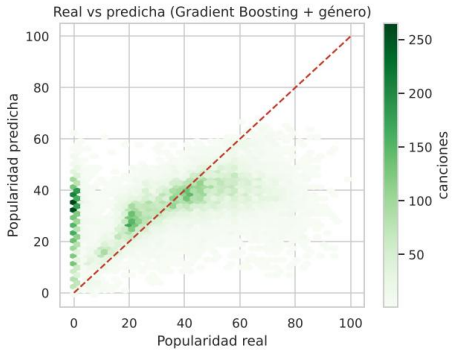


Figura 23. *Popularidad real contra popularidad predicha (Gradient Boosting con género)*



7.4.6 Análisis 3: clasificación del género musical

Paso 17. Experimento A: 112 géneros con Random Forest. Como una canción puede estar en varios géneros, solo se usan las 72,938 canciones que aparecen en un único género. *Singer-songwriter* y *songwriter* se quedan sin canciones exclusivas, así que son 112 géneros. Con solo las 12 características acústicas, el Random Forest obtiene 26.9% de accuracy (0.9% sería el azar), 45.0% de accuracy top-3 y F1 macro de 0.202.

Figura 24. *Clasificación de 112 géneros con Random Forest*

```
In [29]: unicas =
dataset[~dataset['track_id'].duplicated(keep=False)].reset_index(drop=True)
acusticas = ['danceability', 'energy', 'key', 'loudness', 'mode', 'speechiness',
'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo',
'time_signature']
print('Canciones con un solo género:', len(unicas))
unicas['track_genre'].value_counts().describe()[['min', 'mean', 'max']].round(0)

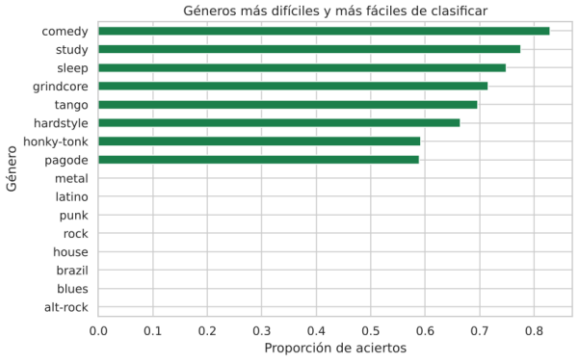
Canciones con un solo género: 72938

Out[29]: min      72.0
mean    651.0
max     999.0
Name: count, dtype: float64

In [30]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
ent, pru = next(GroupShuffleSplit(n_splits=1, test_size=0.2,
random_state=42).split(unicas, groups=unicas['cancion']))
Xa_train, Xa_test = unicas.loc[ent, acusticas], unicas.loc[pru, acusticas]
ya_train, ya_test = unicas.loc[ent, 'track_genre'], unicas.loc[pru,
'track_genre']
rf_a = RandomForestClassifier(n_estimators=100, min_samples_leaf=2, n_jobs=2,
random_state=42).fit(Xa_train, ya_train)
pred_a = np.array(rf_a.predict(Xa_test), dtype=object)
top3 = np.argsort(rf_a.predict_proba(Xa_test), axis=1)[:, -3:]
clases = np.array(rf_a.classes_, dtype=object)
real_a = np.array(ya_test, dtype=object)
print('Géneros:', ya_train.nunique(), '| Acierto al azar:', round(1 /
ya_train.nunique(), 4))
print('Accuracy:', round(accuracy_score(ya_test, pred_a), 3))
print('Accuracy top-3:', round((clases[top3] == real_a[:,
None]).any(axis=1).mean(), 3))
print('F1 macro:', round(f1_score(ya_test, pred_a, average='macro'), 3))

Géneros: 112 | Acierto al azar: 0.0089
Accuracy: 0.269
Accuracy top-3: 0.45
F1 macro: 0.202
```

Figura 25. *Géneros más difíciles y más fáciles de clasificar*



Los géneros que más se reconocen por su sonido son *comedy*, *study*, *sleep* y *grindcore*; los más difíciles (casi 0% de aciertos) son géneros amplios como *rock*, *metal*, *punk*, *house* y *blues*.

Paso 18. Experimento B: 10 géneros con Random Forest y SVM. Se eligen 10 géneros de estilos distintos y se quitan las canciones que aparecen en más de uno de ellos, lo que deja 9,164 canciones casi balanceadas. Para SVM se estandarizan las variables y se prueban combinaciones de `C` y `gamma` con validación cruzada. El Random Forest logra 57.2% de accuracy y el SVM 53.3%.

Figura 26. *Comparación de Random Forest y SVM con 10 géneros*

```
In [32]: from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
elegidos = ['classical', 'country', 'hip-hop', 'heavy-metal', 'jazz', 'pop',
'reggae', 'rock', 'salsa', 'techno']
sub = dataset[dataset['track_genre'].isin(elegidos)]
sub = sub[~sub['track_id'].duplicated(keep=False)].reset_index(drop=True)
print(sub['track_genre'].value_counts().to_dict())
ent_b, pru_b = next(GroupShuffleSplit(n_splits=1, test_size=0.2,
random_state=42).split(sub, groups=sub['cancion']))
Xb_train, Xb_test = sub.loc[ent_b, acusticas], sub.loc[pru_b, acusticas]
yb_train, yb_test = sub.loc[ent_b, 'track_genre'], sub.loc[pru_b, 'track_genre']
print('Canciones:', len(sub), '| Entrenamiento:', len(Xb_train), '| Prueba:',
len(Xb_test))
rf_b = RandomForestClassifier(n_estimators=300, n_jobs=2,
random_state=42).fit(Xb_train, yb_train)
busqueda = GridSearchCV(Pipeline([('escalar', StandardScaler()), ('svm',
SVC())]), {'svm__C': [1, 10], 'svm__gamma': ['scale', 0.2]}, cv=3,
n_jobs=2).fit(Xb_train, yb_train)
print('Mejores parámetros SVM:', busqueda.best_params_)
resultados_gen = pd.DataFrame({'Modelo': ['Random Forest', 'SVM (RBF)'],
                               'Accuracy': [accuracy_score(yb_test,
rf_b.predict(Xb_test)), accuracy_score(yb_test, busqueda.predict(Xb_test))],
                               'F1 macro': [f1_score(yb_test,
rf_b.predict(Xb_test), average='macro'), f1_score(yb_test,
busqueda.predict(Xb_test), average='macro')]}).round(3)
resultados_gen

{'techno': 999, 'heavy-metal': 996, 'salsa': 991, 'jazz': 981, 'country': 961,
'rock': 952, 'classical': 855, 'reggae': 836, 'pop': 829, 'hip-hop': 764}
Canciones: 9164 | Entrenamiento: 7349 | Prueba: 1815
Mejores parámetros SVM: {'svm__C': 10, 'svm__gamma': 'scale'}
```

Out[32]:

	Modelo	Accuracy	F1 macro
0	Random Forest	0.572	0.550
1	SVM (RBF)	0.533	0.521

Paso 19. Matriz de confusión y reporte de clasificación. Con el mejor modelo (Random Forest) se ve que *classical* (F1 = 0.84), *techno* (0.80) y *heavy-metal* (0.77) se reconocen bien, mientras que *rock* (0.17), *pop* (0.30) e *hip-hop* (0.39) se confunden con otros géneros.

Figura 27. *Matriz de confusión con 10 géneros*

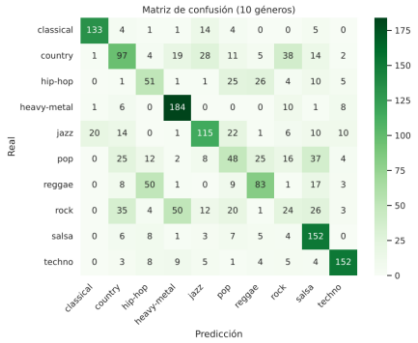
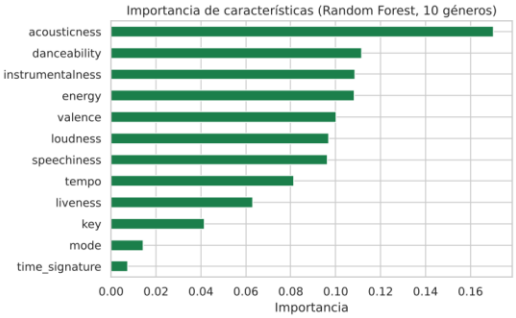


Figura 28. *Reporte de clasificación (precisión, recall y F1 por género)*

```
In [34]: print(classification_report(yb_test, pred_b, digits=2))
```

	precision	recall	f1-score	support
classical	0.86	0.82	0.84	162
country	0.49	0.44	0.46	219
heavy-metal	0.68	0.88	0.77	210
hip-hop	0.37	0.41	0.39	124
jazz	0.62	0.58	0.60	199
pop	0.33	0.27	0.30	177
reggae	0.55	0.48	0.52	172
rock	0.22	0.14	0.17	175
salsa	0.55	0.82	0.66	186
techno	0.81	0.80	0.80	191
accuracy			0.57	1815
macro avg	0.55	0.56	0.55	1815
weighted avg	0.56	0.57	0.56	1815

Figura 29. *Importancia de las características acústicas (10 géneros)*



7.4.7 Análisis 4: reglas de asociación con Apriori

Paso 20. Discretizar las variables. Las características continuas se dividen en terciles (baja, media y alta); `instrumentalness` en vocal, mixta e instrumental; y la popularidad en alta (49 o más, el 25% superior) y baja (19 o menos, el 25% inferior). Con variables ficticias se obtiene una matriz de 89,578 canciones por 30 elementos.

Figura 30. *Discretización de las variables y matriz de transacciones*

```
In [36]: discretas = pd.DataFrame(index=canciones.index)
         for col in ['danceability', 'energy', 'loudness', 'speechiness', 'acousticness',
         'liveness', 'valence', 'tempo']:
             discretas[col] = pd.qcut(canciones[col].rank(method='first'), 3, labels=
         ['baja', 'media', 'alta'])
         discretas['instrumentalness'] = pd.cut(canciones['instrumentalness'], [-0.01,
         0.01, 0.5, 1.0], labels=['vocal', 'mixta', 'instrumental'])
         discretas['explicit'] = np.where(canciones['explicit'] == 1, 'si', 'no')
         q1, q3 = canciones['popularity'].quantile([0.25, 0.75])
         discretas['popularidad'] = np.select([canciones['popularity'] >= q3,
         canciones['popularity'] <= q1], ['alta', 'baja'], 'media')
         print('Popularidad alta: >=', q3, '| baja: <=', q1)
         transacciones = pd.get_dummies(discretas).astype(bool).drop(columns=
         ['popularidad_media', 'explicit_no'])
         print(transacciones.shape)
         transacciones.mean().round(3).sort_values(ascending=False).head(12)

         Popularidad alta: >= 49.0 | baja: <= 19.0
         (89578, 30)

Out[36]: instrumentalness_vocal    0.683
         danceability_baja         0.333
         danceability_alta         0.333
         energy_baja               0.333
         energy_media              0.333
         danceability_media        0.333
         loudness_baja             0.333
         loudness_media            0.333
         loudness_alta             0.333
         speechiness_baja          0.333
         speechiness_media         0.333
         speechiness_alta          0.333
         dtype: float64
```

Paso 21. Algoritmo Apriori programado en el notebook. Como `mlxtend` no estaba disponible, se programó Apriori: se buscan itemsets frecuentes de 1 a 4 elementos con soporte mínimo de 0.02 y solo se combinan los que ya eran frecuentes. Se encontraron 4,778 itemsets frecuentes.

Figura 31. *Implementación del algoritmo Apriori*

In [37]:
```
def apriori_propio(matriz, min_support, max_len):
    n = len(matriz)
    bits = {c: int.from_bytes(np.packbits(matriz[c].values).tobytes(), 'big') for
c in matriz.columns}
    nivel = {}
    for c in matriz.columns:
        s = bits[c].bit_count() / n
        if s >= min_support:
            nivel[(c,)] = (bits[c], s)
    frecuentes = {k: v[1] for k, v in nivel.items()}
    longitud = 1
    while nivel and longitud < max_len:
        longitud += 1
        claves = sorted(nivel)
        nuevos = {}
        for i in range(len(claves)):
            for j in range(i + 1, len(claves)):
                a, b = claves[i], claves[j]
                if a[:-1] != b[:-1]:
                    break
                cand = a + (b[-1],)
                if all(cand[:m] + cand[m + 1:] in nivel for m in range(len(cand)
- 1)):
                    mascara = nivel[a][0] & bits[b[-1]]
                    s = mascara.bit_count() / n
                    if s >= min_support:
                        nuevos[cand] = (mascara, s)
        nivel = nuevos
        frecuentes.update({k: v[1] for k, v in nivel.items()})
    return frecuentes
inicio = time.time()
frecuentes = apriori_propio(transacciones, min_support=0.02, max_len=4)
print('Itemsets frecuentes:', len(frecuentes), '| segundos:', round(time.time() -
inicio, 1))
```

```
Itemsets frecuentes: 4778 | segundos: 0.0
```

Paso 22. Generar y ordenar las reglas. Para cada itemset que contiene la popularidad se calculan *support*, *confidence* y *lift*. Salen 281 reglas hacia popularidad alta y 226 hacia popularidad baja. La popularidad alta aparece en el 25.1% de las canciones y la baja en el 25.7%.

Figura 32. *Reglas hacia popularidad alta (mayores lift)*

In [38]:
```
def reglas_para(consecuente, min_conf=0.0, min_lift=1.0):
    filas = []
    for itemset, sop in frecuentes.items():
        if consecuente in itemset and len(itemset) > 1:
            antecedente = tuple(x for x in itemset if x != consecuente)
            conf = sop / frecuentes[antecedente]
            lift = conf / frecuentes[(consecuente,)]
            if conf >= min_conf and lift >= min_lift:
                filas.append((' + '.join(antecedente), len(antecedente), sop, round(sop
* len(canciones)), conf, lift))
    return pd.DataFrame(filas, columns=['Antecedente', 'Elementos', 'Support',
'Canciones', 'Confidence', 'Lift']).sort_values('Lift',
ascending=False).reset_index(drop=True)
reglas_alta = reglas_para('popularidad_alta')
reglas_baja = reglas_para('popularidad_baja')
print('Reglas hacia popularidad alta (lift > 1):', len(reglas_alta), '| hacia
popularidad baja:', len(reglas_baja))
print('Popularidad alta base:', round(frecuentes[('popularidad_alta',)], 3), '| baja
base:', round(frecuentes[('popularidad_baja',)], 3))
```

```
Reglas hacia popularidad alta (lift > 1): 281 | hacia popularidad baja: 226
Popularidad alta base: 0.251 | baja base: 0.257
```

In [39]:
```
reglas_alta.head(10).round(3)
```

Out[39]:

	Antecedente	Elementos	Support	Canciones	Confidence	Lift
0	explicit_si + instrumentalness_vocal	2	0.025	2270	0.350	1.393
1	instrumentalness_vocal + liveness_media + vale...	3	0.022	1935	0.346	1.377
2	energy_media + instrumentalness_vocal + loudne...	3	0.031	2755	0.342	1.361
3	danceability_alta + instrumentalness_vocal + v...	3	0.025	2266	0.340	1.353
4	acousticness_baja + energy_media + instrumenta...	3	0.020	1823	0.338	1.346
5	energy_media + loudness_alta	2	0.033	2979	0.337	1.342
6	explicit_si	1	0.029	2595	0.337	1.341
7	energy_media + instrumentalness_vocal + livene...	3	0.028	2542	0.329	1.309
8	instrumentalness_vocal + liveness_media + loud...	3	0.025	2247	0.328	1.306
9	acousticness_media + instrumentalness_vocal + ...	3	0.030	2705	0.327	1.301

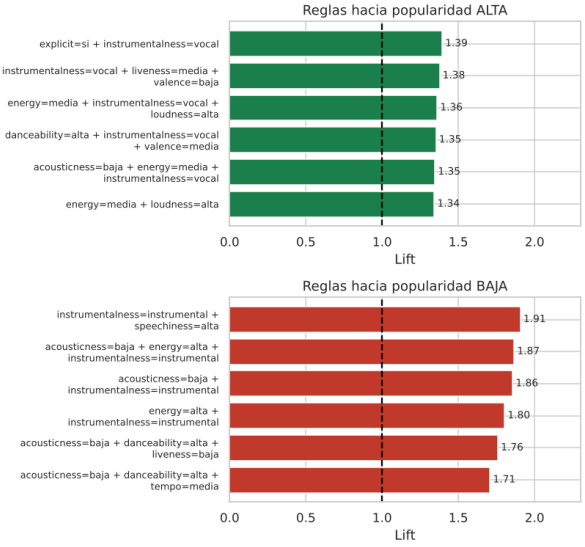
Figura 33. *Reglas hacia popularidad baja (mayores lift)*

```
In [40]: reglas_baja.head(10).round(3)
```

Out[40]:

	Antecedente	Elementos	Support	Canciones	Confidence	Lift
0	instrumentalness_instrumental + speechiness_alta	2	0.024	2161	0.490	1.907
1	acousticness_baja + energy_alta + instrumental...	3	0.024	2112	0.480	1.866
2	acousticness_baja + instrumentalness_instrumental	2	0.037	3325	0.477	1.856
3	energy_alta + instrumentalness_instrumental	2	0.028	2474	0.464	1.803
4	acousticness_baja + danceability_alta + livene...	3	0.021	1855	0.452	1.759
5	acousticness_baja + danceability_alta + tempo_...	3	0.025	2264	0.439	1.707
6	danceability_alta + instrumentalness_instrumental	2	0.024	2190	0.438	1.702
7	instrumentalness_instrumental + tempo_media	2	0.030	2682	0.425	1.652
8	acousticness_baja + danceability_alta	2	0.040	3551	0.409	1.589
9	instrumentalness_instrumental + liveness_baja	2	0.029	2590	0.392	1.524

Figura 34. *Lift de las reglas más fuertes hacia popularidad alta y baja*



Las asociaciones hacia popularidad alta son débiles: la mejor (`explicit_si + instrumentalness_vocal`) tiene lift de 1.39 y confianza de 35.0% contra 25.1% base. Hacia popularidad baja son más fuertes: `instrumentalness_instrumental + speechiness_alta` tiene lift de 1.91 y confianza de 49.0%.

7.5 Análisis adicionales

Además de los análisis principales, en el notebook se realizaron los siguientes análisis para el mismo problema, que ya se mostraron en los pasos anteriores:

- **Popularidad con y sin género (problema 2):** se compararon cuatro modelos con dos conjuntos de variables. Pasar de $R^2 = 0.091$ a 0.285 al agregar el género mostró que la información clave no está en el audio.
- **Clasificación con 112 y con 10 géneros (problema 3):** se probó el problema completo y una versión reducida y balanceada, comparando Random Forest contra SVM y reportando accuracy, top-3, F1, matriz de confusión e importancia de variables.
- **Elección de k en la segmentación (problema 1):** además del método del codo se usó el coeficiente de silueta, y se revisaron los resultados con k = 6 y k = 7.
- **Reglas hacia popularidad alta y baja (problema 4):** se generaron las dos familias de reglas para poder contrastar qué rasgos acompañan el éxito y cuáles acompañan la baja popularidad.
- **Visualizaciones:** histogramas, matriz de correlación, popularidad por género, mapa de calor de perfiles, proyección PCA, gráfica de real contra predicha y matriz de confusión.

8. Resultados

8.1 Resultados obtenidos

En la Tabla 7 se resumen los resultados principales de los cuatro análisis. Después se listan uno por uno.

Tabla 7. *Resumen de resultados principales*

ANÁLISIS	MÉTRICA	RESULTADO
1. Segmentación (K-Means, k = 5)	Silueta / varianza de PCA	0.179 / 47.9% con 2 componentes
2. Popularidad con audio y metadatos	Mejor R^2 / MAE	0.091 (Gradient Boosting) / 17.1 puntos (base: 18.8)
2. Popularidad agregando género	Mejor R^2 / MAE	0.285 (Gradient Boosting) / 13.95 puntos
3. Género, 112 clases (Random Forest)	Accuracy / top-3 / F1 macro	26.9% / 45.0% / 0.202
3. Género, 10 clases	Accuracy RF / SVM	57.2% / 53.3%
4. Asociación (Apriori)	Reglas / mayor lift	281 hacia alta (1.39) y 226 hacia baja (1.91)

1. **Cinco segmentos de canciones** interpretables: Hablado (1.2% de las canciones), Bailable y alegre (37.7%), Suave y acústico (23.1%), Intenso y oscuro (29.9%) e Instrumental y ambiental (8.1%), con una silueta de 0.179 y un 47.9% de la varianza visible en 2 dimensiones.
2. **Un recomendador por contenido** que, para una canción dada, devuelve las canciones más parecidas de su mismo segmento (por ejemplo, otras versiones de *Despacito* y canciones de otros géneros con sonido similar).
3. **La popularidad casi no se predice con audio y metadatos:** R^2 máximo de 0.091 y MAE de 17.1 puntos, contra 18.8 del modelo base.
4. **Al agregar el género la predicción mejora de forma importante pero limitada:** R^2 = 0.285 y MAE de 13.95; el género aporta casi la mitad de la importancia del modelo y aun así el 71.5% de la variación queda sin explicar.
5. **El género se reconoce parcialmente por el sonido:** 26.9% de accuracy con 112 géneros (45.0% si se aceptan los 3 más probables) y 57.2% con 10 géneros, donde *classical*, *techno* y *heavy-metal* se distinguen bien y *rock*, *pop* e *hip-hop* se confunden.
6. **Reglas de asociación débiles hacia la popularidad alta** (lift máximo de 1.39: letra explícita y con voz) **y algo más fuertes hacia la popularidad baja** (lift de 1.91: canciones instrumentales).
7. **Resultado deseado no alcanzado:** se esperaba poder predecir la popularidad con precisión a partir del audio; los datos demuestran que no es posible con las variables disponibles.

8.2 Cómo se podrían aprovechar los resultados

Tabla 8. *Aplicación de cada resultado*

RESULTADO	CÓMO APROVECHARLO
Segmentos de canciones	Crear listas de reproducción automáticas por tipo de sonido o actividad (entrenar, dormir, fiesta), organizar el catálogo y separar el contenido hablado de la música.
Recomendador por contenido	Recomendar canciones nuevas que todavía no tienen historial de reproducciones (problema de arranque en frío) y complementar a los recomendadores basados en el comportamiento de los usuarios.

RESULTADO	CÓMO APROVECHARLO
Modelo de popularidad	Usarlo solo como un puntaje de apoyo para priorizar lanzamientos o promociones, sabiendo que explica poco; el género sirve como referencia de la popularidad esperada.
Clasificador de género	Etiquetar automáticamente canciones nuevas o sin género, detectar etiquetas dudosas y sugerir los 3 géneros más probables para que una persona decida.
Reglas de asociación	Dar pistas a productores y disqueras sobre qué rasgos acompañan a los éxitos (voz, letra explícita) y reconocer que la música instrumental necesita otra estrategia, porque rara vez alcanza popularidad alta.

8.3 Otras opciones y soluciones aplicables

- **Mejorar la predicción de popularidad con más datos:** agregar el artista (por ejemplo, su número de seguidores), la fecha de lanzamiento, la presencia en listas de reproducción o la letra (con procesamiento de lenguaje natural). Es la solución más prometedora, ya que el audio por sí solo no alcanza.
- **Usar XGBoost o LightGBM** con ajuste de hiperparámetros (se propuso en la actividad 3.1 y no estaba instalado), y probar convertir la popularidad en un problema de clasificación (popular o no) con métricas como el AUC.
- **Probar otros algoritmos de agrupamiento:** mezclas gaussianas, agrupamiento jerárquico o DBSCAN, que permiten grupos de formas y tamaños distintos, y combinarlos con *embeddings* de audio.
- **Clasificar por familias de géneros** (rock, electrónica, latina, etc.) o con etiquetas múltiples, porque una canción puede pertenecer a varios géneros y muchos géneros del conjunto son muy cercanos entre sí.
- **Usar FP-Growth o Eclat** en lugar de Apriori e incluir el género y el año de lanzamiento como elementos de las reglas.

9. Conclusiones

9.1 Conclusión general

El proyecto mostró que la minería de datos permite contestar las cuatro preguntas del caso de estudio, pero con resultados muy distintos. Las características de audio bastan para **describir y organizar** el catálogo: se obtuvieron cinco segmentos de canciones con sentido musical, un recomendador por similitud y un clasificador capaz de distinguir géneros muy diferentes entre sí. En cambio, **no bastan para predecir la popularidad**: con audio y metadatos el R^2 fue de apenas 0.09 y, aun agregando el género, llegó a 0.285. Las reglas de asociación confirmaron lo mismo, pues ninguna combinación de rasgos de audio multiplica por más de 1.4 la probabilidad de ser popular.

Además del resultado, el trabajo dejó aprendizajes de método: separar los datos por grupos para evitar que una misma canción aparezca en entrenamiento y prueba, estandarizar antes de usar distancias, comparar siempre contra un modelo base, probar el problema completo y una versión reducida, y reportar con honestidad los resultados bajos. Para continuar, lo más valioso sería incorporar información que no está en el audio (artista, fecha de lanzamiento, promoción y comportamiento de los usuarios).

9.2 Conclusiones individuales

Luis Rolando Aguilar Ortiz. Me enfoqué en la segmentación con K-Means. Aprendí que elegir el número de clusters no siempre es automático: el método del codo no mostró un punto claro y la silueta favorecía k = 2, así que tuvimos que apoyarnos en la interpretación de los grupos para decidir por k = 5. Me pareció muy útil ver que los segmentos coinciden con géneros que el algoritmo nunca vio, y que con ellos se puede armar un recomendador para canciones nuevas.

Yael Abiezer García Guerra. Mi aportación se centró en la predicción de la popularidad. El resultado más importante para mí fue comprobar que un modelo complejo no siempre mejora lo que se espera: con audio el R^2 fue de solo 0.09 y el género fue la variable que más ayudó. Aprendí a comparar contra un modelo base, a separar por grupos para no inflar los resultados y a explicar con claridad qué significa un R^2 bajo.

Perla Natalie González Treviño. Trabajé en la clasificación de géneros. Me llamó la atención que el sonido sirve para reconocer géneros muy marcados, como clásica, techno y metal, pero que rock, pop e hip-hop se confunden con otros. Aprendí a leer una matriz de confusión y un reporte de clasificación, y a usar la accuracy top-3 para entender que, aunque el modelo falle en el primer intento, muchas veces el género correcto queda entre los más probables.

Axel Ivan Verastegui Patiño. Me tocó el análisis de reglas de asociación. Aprendí que para usar Apriori hay que transformar las variables continuas en categorías y que un lift mayor a 1 no es suficiente para decir que algo importa: aquí los valores fueron bajos. Lo más interesante fue ver que lo que más se asocia con la popularidad baja es que la canción sea instrumental, y que no existe una combinación acústica que garantice el éxito.

Jaime Cesar Zermeño Pereyra. Participé en la comprensión y la limpieza de los datos. Descubrimos que había canciones repetidas en varios géneros, mediciones inválidas y que los géneros estaban balanceados; sin atender esto, los resultados se habrían inflado. Considero que la preparación de datos fue tan importante como los modelos, y que este proyecto mostró cómo se sigue el proceso completo de minería de datos, desde la pregunta de negocio hasta las conclusiones.

10. Referencias

Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. En J. B. Bocca, M. Jarke, & C. Zaniolo (Eds.), *Proceedings of the 20th International Conference on Very Large Data Bases* (pp. 487-499). Morgan Kaufmann.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273-297. https://doi.org/10.1007/BF00994018

Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine, 17*(3), 37-54. https://doi.org/10.1609/aimag.v17i3.1230

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189-1232. https://doi.org/10.1214/aos/1013203451

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3.a ed.). Morgan Kaufmann.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A, 374*(2065), Artículo 20150202. https://doi.org/10.1098/rsta.2015.0202

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. En L. M. Le Cam & J. Neyman (Eds.), *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281-297). University of California Press.

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56-61). https://doi.org/10.25080/Majora-92bf1922-00a

Pandya, M. (s.f.). *Spotify Tracks Dataset* [Conjunto de datos]. Hugging Face. https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53-65. https://doi.org/10.1016/0377-0427(87)90125-7

Spotify. (2026, 22 de octubre). *Spotify reports third quarter 2026 earnings*. Spotify Newsroom. https://newsroom.spotify.com/2026-10-22/spotify-q3-2026-earnings/